

我扫完了，项目可以概括为：**面向 CS 论文的 PDF-to-Markdown + RAG 摘要发布自动化框架**。它不只是 OCR 或格式转换，而是把论文解析、结构清洗、知识库问答、多角色摘要校验、图片抽取、中英输出和 GitHub 发布串成了一条流水线。

项目亮点

1.

高质量 PDF/论文解析流水线

基于 marker，支持 PDF 文本抽取、版面识别、阅读顺序恢复、表格格式化、公式转换、代码块识别、图片抽取和 Markdown 后处理。相关实现集中在 `marker/convert.py`。

2.

3.

OCR 模块较完整

OCR 不是简单一刀切调用，而是先检测文本行，再通过启发式判断页面是否需要 OCR。支持 surya OCR，也支持 ocrmypdf/Tesseract 后端；依赖里也明确包含 surya-ocr。相关文件包括 `marker/ocr/recognition.py`、`marker/ocr/heuristics.py`、`docs/install_ocrmypdf.md`。

4.

5.

轻量级多角色 Agent/RAG 工作流

你记得的“多个模型扮演不同角色”是存在的。代码里定义了 SUMMARIZER、QUESTIONER、ANSWERER 三个角色：先总结论文，再由提问者检查摘要完整性，最后由回答者结合知识库生成更完整摘要。

严格说，这可以写成 **role-based multi-agent workflow** 或 **multi-role RAG agent pipeline**，虽然它不是复杂自治 Agent，但作为“多角色协作式摘要代理”是合理的。核心在 `green_work/main_struct/rag/agent.py` 和 `green_work/main_struct/rag/prompt.py`。

6.

7.

集成 QAnything 知识库问答

每篇论文会被整理成临时文本，创建知识库、上传文件、等待解析、基于知识库问答生成摘要，然后删除知识库，避免只靠长上下文硬塞全文。相关实现见 `green_work/main_struct/rag/create_kb.py`、`upload_files.py`、`question_request.py`。

8.

9.

论文内容结构化过滤

Markdown 生成后，系统会自动提取摘要、引言和方法部分，并过滤 reference、acknowledgment、conclusion 等低价值段落，减少摘要噪声。相关文件是 `green_work/main_struct/file_processor/filter.py`。

10.

11.

自动生成项目/论文列表型 README

系统会把论文标题、代表性图片和摘要写入 Markdown 表格，适合生成论文合集、调研仓库或阅读索引。实现见 `green_work/main_struct/file_processor/output.py`。

12.

13.

图片自动抽取与代表图筛选

会从论文 introduction 中识别图片引用，并选择较大尺寸图片作为展示图，增强生成 README 的可读性。相关文件在 `green_work/main_struct/picture_processor`。

14.

15.

支持中英文摘要发布

项目包含中文翻译流程，可生成 `README_CN.md`，适合双语论文资源库。相关文件在 `green_work/translation`。

16.

17.

端到端自动发布到 GitHub

`green_work/main.py` 会拉取目标仓库、复制图片、写入 `README/README_CN`，最后 commit 并 push，实现从 PDF 文件夹到 GitHub 论文索引的自动化发布。

18.

可直接用于项目描述的版本

这个项目是一个面向计算机科学论文的自动化 PDF 解析与论文摘要生成框架。系统基于 Marker 完成 PDF 到 Markdown 的高质量转换，集成 OCR、版面分析、阅读顺序恢复、表格/公式/图片抽取等能力；随后通过 QAnything 构建临时

知识库，并设计了 Summarizer、Questioner、Answerer 三个角色组成的轻量级多角色 RAG 工作流，对论文内容进行摘要、校验和补全。项目还支持自动筛选论文代表图、中英文摘要生成，以及将结果写入 README 并同步到 GitHub 仓库，实现从论文 PDF 批量处理到可发布论文索引的端到端自动化流程。

一句话版

基于 OCR、文档结构解析和多角色 RAG Agent 的 CS 论文自动摘要与 GitHub 知识库发布系统。